

Nonlinear Trajectory Optimization and Differential Dynamic Programming

Optimization and Learning for Robot Control

RCI Lab

Kyung Hee University

February 21, 2026

From Convex MPC to Nonlinear TrajOpt

Linear MPC (Convex case)

$$x_{k+1} = Ax_k + Bu_k$$

$$\min_{\{x_k, u_k\}} \sum_{k=0}^{N-1} \ell(x_k, u_k) + \ell_f(x_N)$$

If ℓ is quadratic $\rightarrow$ QP $\rightarrow$ convex.

Nonlinear case

$$x_{k+1} = f(x_k, u_k)$$

$\rightarrow$ Problem becomes non-convex.

Nonlinear Trajectory Optimization Problem

$$\min_{\{x_k, u_k\}} \sum_{k=0}^{N-1} \ell(x_k, u_k) + \ell_f(x_N)$$

subject to

$$x_{k+1} = f(x_k, u_k)$$

Assume:

- f, ℓ are twice continuously differentiable
- Enables second-order Taylor expansion

Dynamic Programming Principle

Define cost-to-go:

$$V_k(x) = \min_{u_k} [\ell(x, u_k) + V_{k+1}(f(x, u_k))]$$

Define Q-function:

$$Q_k(x, u) = \ell(x, u) + V_{k+1}(f(x, u))$$

Then:

$$V_k(x) = \min_u Q_k(x, u)$$

DDP $\simeq$ apply second-order expansion to Q_k .

Second Order Expansion of Q

Let perturbations:

$$x = \bar{x} + \delta x$$

$$u = \bar{u} + \delta u$$

Second order expansion:

$$Q(x, u) \approx Q + Q_x \delta x + Q_u \delta u + \frac{1}{2} \begin{bmatrix} \delta x \\ \delta u \end{bmatrix}^T \begin{bmatrix} Q_{xx} & Q_{xu} \\ Q_{ux} & Q_{uu} \end{bmatrix} \begin{bmatrix} \delta x \\ \delta u \end{bmatrix}$$

Optimal Control Update

Optimality condition:

$$\frac{\partial Q}{\partial u} = 0$$

$$Q_u + Q_{ux}\delta x + Q_{uu}\delta u = 0$$

Solve for δu :

$$\delta u = -Q_{uu}^{-1}Q_u - Q_{uu}^{-1}Q_{ux}\delta x$$

Define:

$$k = -Q_{uu}^{-1}Q_u$$

$$K = -Q_{uu}^{-1}Q_{ux}$$

Control law:

$$\delta u = k + K\delta x$$

Value Function Update

Plug back into Q expansion.

After algebra:

$$V_x = Q_x - Q_{xu} Q_{uu}^{-1} Q_u$$

$$V_{xx} = Q_{xx} - Q_{xu} Q_{uu}^{-1} Q_{ux}$$

Backward pass computes:

$$k_k, K_k, V_x, V_{xx}$$

Forward pass simulates new trajectory.

Derivatives of Q

Recall:

$$Q(x, u) = \ell(x, u) + V_{k+1}(f(x, u))$$

Using chain rule:

$$Q_x = \ell_x + f_x^T V'_x$$

$$Q_u = \ell_u + f_u^T V'_x$$

Second derivatives:

$$Q_{xx} = \ell_{xx} + f_x^T V'_{xx} f_x + \sum_i V'_{x_i} f_{xx}^{(i)}$$

Similarly:

$$Q_{ux} = \ell_{ux} + f_u^T V'_{xx} f_x + \sum_i V'_{x_i} f_{ux}^{(i)}$$

Matrix Calculus: Vectorization Trick

For matrix $A \in \mathbb{R}^{m \times n}$

$$\text{vec}(A) = \begin{bmatrix} a_{11} \\ \vdots \\ a_{m1} \\ a_{12} \\ \vdots \end{bmatrix}$$

Kronecker identity:

$$\text{vec}(ABC) = (C^T \otimes A)\text{vec}(B)$$

Special cases:

$$\text{vec}(AB) = (I \otimes A)\text{vec}(B) = (B^T \otimes I)\text{vec}(A)$$

Why Vectorization Matters

When differentiating matrix functions:

$$\frac{\partial}{\partial X}(AXB)$$

Vectorize:

$$\text{vec}(AXB) = (B^T \otimes A)\text{vec}(X)$$

Thus Jacobian becomes explicit matrix form.

Essential for:

- Second order derivatives in DDP
- Automatic differentiation structure
- Efficient implementation

DDP Algorithm Summary

Initialize trajectory $\{x_k, u_k\}$

Repeat:

1. Backward pass:

$$k_k, K_k, V_x, V_{xx}$$

2. Forward rollout:

$$u_k^{new} = u_k + \alpha k_k + K_k(x_k^{new} - x_k)$$

3. Line search on α

Converges quadratically near optimum

- $DDP \simeq$ Newton method in trajectory space
- iLQR = Gauss-Newton approximation
- Related to SQP
- Widely used in robotics (humanoids, MPC warm-start)